

Etapa 3 — Implementação do Classificador Naive Bayes em SQL

Autor Marcus Vinícius Santos de Almeida	Data de entrega 11/09/2026
Banco de dados DuckDB — <code>dados/classificador.db</code>	Entrada de treino <code>etapa2_dados_treinamento.csv</code> (1.000.000 de linhas)
Artefatos <code>sql/classificador_naive_bayes.sql</code> <code>sql/rodar_classificador.py</code>	

Este relatório explica a arquitetura do código para apoiar a defesa oral — não repete o SQL linha a linha. A lógica SQL é essencialmente a mesma da versão anterior (mesmas 5 views, mesma fórmula de Laplace, mesma normalização em log); o que mudou é (a) a migração de SQLite para DuckDB e (b) os dados de entrada, agora $N = 1.000.000$ registros.

1 Por que DuckDB

SQLite é um motor **transacional**, otimizado para muitas escritas pequenas e intercaladas. Este projeto tem o padrão oposto: os dados são escritos **uma vez** (a importação do CSV) e depois só sofrem **agregação** repetida (`GROUP BY` , `JOIN` nas views) — exatamente o padrão para o qual um motor **analítico e colunar** como o DuckDB foi desenhado. Com $N = 1.000.000$ (e `treino_longo` chegando a 6.000.000 de linhas), esse descompasso deixou de ser hipotético.

A troca **mantém** as vantagens do SQLite já documentadas antes:

- **Roda sem servidor.** Um único arquivo (`classificador.db`); sem processo, porta ou senha — só `import duckdb` .
- **Fácil de testar.** O script (re)cria o banco do zero, importa o CSV, roda o classificador e confere o resultado em segundos.
- **SQL padrão suficiente.** `CREATE VIEW` , `JOIN` , `GROUP BY` , CTEs, `UNION ALL` , `CASE` , `LN / EXP` — tudo nativo no DuckDB, como já era no SQLite.

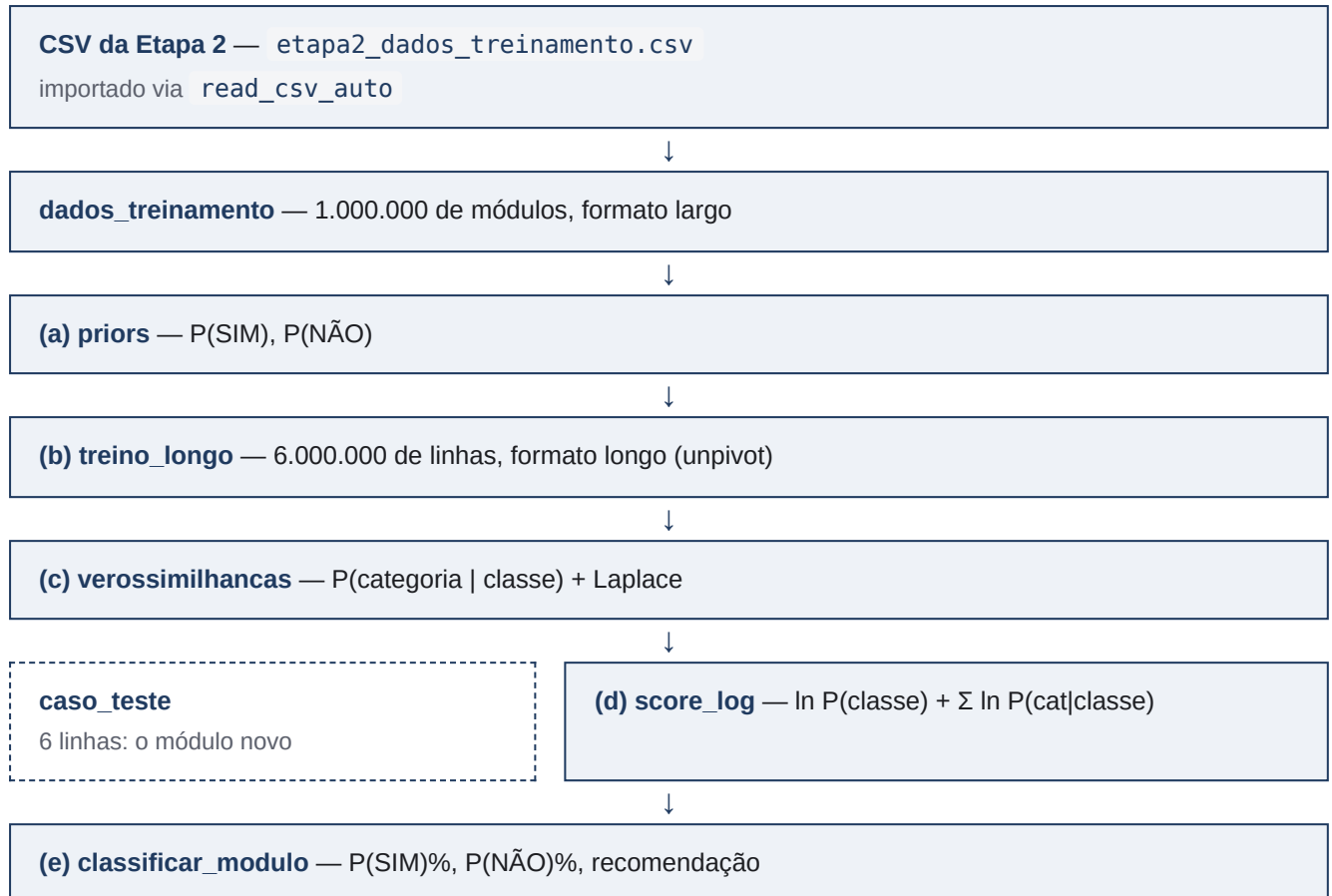
E **ganha** três coisas na troca:

1. **LN / EXP nativos, sem fallback.** Diferente do SQLite, não é preciso nenhum código de contingência em Python para garanti-las (Seção 4).
2. **Importação de CSV mais direta.** `read_csv_auto('arquivo.csv')` faz leitura, inferência de tipos e carga numa única instrução, em vez do laço manual de `INSERT` que o SQLite exigia.
3. **Interface visual nativa** (`duckdb -ui`) — útil para inspecionar as views ao vivo durante a apresentação, sem ferramenta externa.

Custo assumido

Assim como o SQLite, o DuckDB não tem `PIVOT` nativo — o "unpivot" continua feito à mão com `UNION ALL` (Seção 3.2), o que na verdade deixa a lógica mais explícita.

2 Fluxo de dados (do CSV ao veredito)



3 As views, uma a uma

3.1 `priors` — probabilidade a priori $P(\text{classe})$

Conta quantos módulos de treino são SIM e quantos são NÃO e divide pelo total.

Resultado com a massa atual: $P(\text{NÃO}) = 650.000/1.000.000 = 0,6500$ e $P(\text{SIM}) = 350.000/1.000.000 = 0,3500$. Equivalente à proporção com $N=150$ (então 34,7%/65,3%) — o gerador fixa `PROPORCAO_DEFEITO = 0,35` via seleção top-k, independentemente de como as features são sorteadas; com N maior, o arredondamento bate exatamente em 35,0%.

3.2 `treino_longo` — "unpivot" das 6 categorias

Para calcular a verossimilhança de todas as features com **uma** consulta em vez de seis, empilhamos os seis recortes com `UNION ALL`, chegando ao formato longo:

`(modulo_id, defeito, feature, categoria)` — 1.000.000 de módulos $\times$ 6 features = **6.000.000 de linhas**.

3.3 `verossimilhanças` — $P(\text{categoria} | \text{classe})$ com suavização de Laplace

$P(\text{feature} = \text{categoria} | \text{classe}) = (\text{contagem} + 1) / (\text{total_da_classe} + k)$, com $k = 3$, igual para as 6 features.

Por que o "+1" e o "+k" (Laplace)

Sem eles, uma categoria nunca vista numa classe teria contagem 0 e probabilidade 0 — e como o Naive Bayes soma os logs, **um único zero anula a classe inteira**. O Laplace adiciona meia folga a cada categoria: nenhuma probabilidade é exatamente 0 nem 1, e o efeito sobre categorias bem povoadas é desprezível.

Exemplo concreto: `complexidade = alto` na classe SIM aparece 83.046 vezes; $P = (83.046 + 1) / (350.000 + 3) = 83.047/350.003 \approx 0,2373$.

3.4 `score_log` — somar logaritmos em vez de multiplicar

Por que log

Multiplicar 7 números pequenos (prior + 6 verossimilhanças) gera valores minúsculos — underflow. Tomando log, o produto vira soma:

$$\log \text{score}(\text{classe}) = \ln P(\text{classe}) + \sum_i \ln P(\text{cat}_i | \text{classe})$$

3.5 `classificar_modulo` — normalização e recomendação

$$P(\text{SIM}) = \exp(\log_sim) / (\exp(\log_sim) + \exp(\log_nao))$$

Subtraímos `log_max = max(log_sim, log_nao)` dos dois expoentes antes do `exp()` por estabilidade numérica — não muda o resultado.

Condição	Recomendação
P(SIM) > 50%	ALTO RISCO — revisão de código e testes adicionais
caso contrário	BAIXO RISCO — pode seguir o fluxo normal

4 Funções LN e EXP no DuckDB

O DuckDB traz `ln()` e `exp()` **nativas**, sem exigir nenhuma flag de build — confirmado com `SELECT ln(1), exp(0)`.

Diferente da versão anterior (SQLite), não é preciso nenhum código de contingência em Python (`connection.create_function`) para garantir essas funções — o `.sql` roda direto em qualquer instalação padrão do DuckDB. Essa seção existia antes por causa da possibilidade — remota, mas real no SQLite — de um build sem `SQLITE_ENABLE_MATH_FUNCTIONS` ; no DuckDB esse cenário não existe, e o código correspondente foi removido de `rodar_classificador.py`.

5 Teste de ponta a ponta

`sql/rodar_classificador.py` recria o banco, roda o SQL e classifica **dois casos de exemplo mínimos** — apenas um *smoke test*. Os 5+ casos formais e a análise crítica são da Etapa 4.

Caso	Perfil (6 categorias)	P(SIM)	P(NÃO)	Recomendação
<code>exemplo_alto_risco</code>	complexidade alto, loc baixo, autores alto, churn alto, imports alto, cobertura baixo	96,49%	3,51%	ALTO
<code>exemplo_baixo_risco</code>	complexidade baixo, loc medio, autores baixo, churn baixo, imports baixo, cobertura alto	4,41%	95,59%	BAIXO

Verificações automáticas

- As duas probabilidades somam **100,00%** nos dois casos.
- **6/6 features** casaram com a tabela de verossimilhanças.
- O veredito bate com a intuição do domínio.

Log-scores: `exemplo_alto_risco` → ln-score SIM -8,8923 vs. NÃO -12,2072 | `exemplo_baixo_risco` → ln-score SIM -9,9617 vs. NÃO -6,8849.

6 Como executar

```
python3 sql/rodar_classificador.py
```

O script é idempotente. Para inspecionar o modelo manualmente: `duckdb dados/classificador.db`
`"SELECT * FROM verossimilhanças;"` — ou, para uma interface visual local (útil na apresentação):
`duckdb -ui dados/classificador.db`.

Nota sobre o uso de IA

O SQL e o runner foram escritos em diálogo com uma IA generativa; cada view foi revisada e testada (contagens conferidas, soma das verossimilhanças = 1, probabilidades somando 100%). Todo o código é comentado bloco a bloco e é defensável oralmente.